OBJECT ORIENTED ANALYSIS AND DESIGN (CT 651)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 2: Object Oriented Analysis

Syllabus Hours: 8 hrs | Marks Weight: 14 marks

CHAPTER 2 SYLLABUS TOPICS

- 2.1 Building Conceptual Model
- 2.2 Adding Associations and Attributes
- 2.3 Representation of System Behavior

SECTION 1: Building Conceptual Model — Domain Modeling & Conceptual Classes

Exam Ref: 2082 Kartik Q3 [1+5], 2081 Chaitra Q4 [4+6], 2081 Ashwin Q3 [1+6], 2080 Chaitra Q3 [7], 2079 Ashwin Q3, 2079 Chaitra Q4, 2078 Chaitra Q4 [6+4], 2078 Baishakh Q3, 2072 Ashwin Q3

Q1. What are the different strategies to find conceptual classes in a problem domain? Explain category list method with example. [1+5] (2082 Kartik)

Answer:

Domain Modeling / Conceptual Model

A Domain Model (also called Conceptual Model) is a visual representation of conceptual classes (real-world concepts, objects, or things) in the problem domain. It is the most important artifact of OOA. It shows: (1) Conceptual classes with their attributes, (2) Associations between classes, (3) Multiplicity of associations. It does NOT show methods/operations (unlike a design class diagram).

Strategies to Find Conceptual Classes

Strategy	Description	When to Use
1. Category List Method (Noun-Phrase List)	Use a pre-defined checklist of categories of things typically found in domains. For each category, identify if anything in the problem domain fits.	Best starting point — systematic and thorough.
2. Noun Phrase Identification	Extract all noun phrases from the problem description text. Each noun/noun phrase is a candidate conceptual class.	Fast — good for scenario descriptions.
3. Use Case Based Analysis	Examine each use case scenario and identify objects that participate in it.	When use cases are already defined.

Category List Method — Detailed Explanation

The Category List is a predefined checklist of common categories of conceptual classes found in most business domains. For each category, you check whether the system description contains anything that belongs to that category.

Category	Description	Example (Library System)
Physical or tangible objects	Things you can physically touch.	Book, Shelf, Library Card, Computer Terminal
Specifications / descriptions	Descriptions or specifications of items.	BookDescription (title, ISBN, author, publisher)
Places	Physical or logical locations.	Library, Branch, Reading Room

Transactions	Business events, exchanges, or operations.	Loan, Fine, Purchase, Reservation
Transaction line items	Items or records within a transaction.	LoanItem, CheckoutRecord
Roles of people	Roles played by people in the domain.	Librarian, Member, Student, Faculty
Containers	Physical or informational containers.	Catalog, Inventory, BookShelf
Things in a container	Items contained within a container.	Book (in Catalog), Item (in Inventory)
Other systems	External systems the current system interacts with.	PaymentGateway, StudentDatabase
Abstract concepts	Intangible concepts used in the domain.	Membership, Policy, Subscription
Organizations	Groups or companies.	Library, PublishingHouse, University
Events	Occurrences or happenings.	BookReturn, OverdueNotification
Rules / Policies	Business rules governing operations.	LoanPolicy, FineCalculationRule
Catalogs	Lists of items or records.	BookCatalog, MemberCatalog
Records of finance	Financial records and receipts.	FineReceipt, MembershipFeeRecord

Example — Category List for Airline Reservation System:

Category	Conceptual Class Identified
Physical objects	Aircraft, Airport, Gate, Terminal
Transactions	Reservation, Booking, Cancellation, Payment
Transaction line items	BookingItem, SeatAssignment
Roles of people	Passenger, Pilot, FlightAttendant, GroundStaff
Places	Airport, City, Country
Containers	Flight (contains passengers), AircraftCabin
Organizations	Airline, AirportAuthority
Events	Departure, Arrival, Delay, Cancellation
Specifications	FlightSchedule, FareClass, SeatType
Other systems	PaymentGateway, CheckInSystem, BaggageSystem

Q2. Define domain modeling. List the candidate conceptual class for the online cable car ticket booking system by using the Category List method. [1+6] (2081 Ashwin Back)

Answer:

Domain Modeling — Definition

A Domain Model is a visual representation of the important business concepts (conceptual classes) of the problem domain with their associations and attributes. It serves as a conceptual map of the domain and bridges the gap between the problem space and the software solution. In UML, it is shown as a class diagram without methods, focusing on real-world entities.

Steps to Create a Domain Model:

- Step 1: Identify conceptual classes using category list or noun phrase method.
- Step 2: Draw classes as rectangles with class names.
- Step 3: Add associations between related classes (with association names).

- Step 4: Add attributes (simple data values belonging to each class).
- Step 5: Add multiplicities to associations.

Candidate Conceptual Classes for Online Cable Car Ticket Booking System:

Category	Conceptual Class
Physical objects	CableCar, Station, CableCarCabin, Route
Transactions	Booking, Ticket, Payment, Cancellation, Refund
Transaction line items	BookingItem, TicketItem
Roles of people	Passenger, Operator, Administrator, StationStaff
Places	Station (Start/End), Route, TouristDestination
Containers	CableCar (contains Passengers), Schedule
Specifications	TicketType (adult/child/senior), FareClass, RouteSpecification
Events	Departure, Arrival, WeatherDelay, EmergencyStop
Abstract concepts	Reservation, Availability, Capacity
Organizations	CableCarCompany, TourismAuthority
Records of finance	PaymentReceipt, RefundRecord, RevenueRecord
Other systems	PaymentGateway, SMSNotificationSystem, WeatherSystem

---

Q3. Identify the conceptual classes and its supportive attributes for the given scenario and draw domain model for the system. [7] (2080 Chaitra)

Scenario: A customer arrived at the checkout counter with items to purchase. The Cashier uses the POS system to record each item. This system presents running total and line item details. The Customer enters payment information which system validates and records. Then system updates the inventory. The customer receives receipt from the system and leaves the store with the purchased item.

Answer:

Conceptual Classes Identified — POS System:

Conceptual Class	Attributes	Description
Sale	saleID, date, time, total, status	Main transaction recording the purchase.
SaleLineItem	quantity, subtotal	Each individual item recorded in the sale.
ProductDescription	description, price, itemID, category	Description/specification of a product.
Payment	amount, paymentType, paymentDate	Payment made by customer for the sale.
Receipt	receiptID, date, total, tax	Proof of purchase given to customer.
Customer	customerID, name, contact	Person making the purchase.
Cashier	cashierID, name, shift	Employee operating the POS system.
Store	storeID, name, address	Physical store where sale takes place.
Register	registerID, location	POS terminal used for recording sales.
Inventory	inventoryID, location	Stock tracking system updated after sale.
Item	itemID, name, price, quantity	Physical product being purchased.

Domain Model Description — POS System:

- Sale contains 1..* SaleLineItems (composition — line items cannot exist without a sale).
- Each SaleLineItem is associated with 1 ProductDescription.
- Sale is associated with 1 Payment.

- Sale generates 1 Receipt.
- Cashier records 0..* Sales.
- Customer makes 1..* Sales.
- Register records 0..* Sales.
- Sale updates Inventory (dependency relationship).
- Inventory contains 0..* Items.

Q4. Create a category list for Airline reservation application and explain how you would use it to identify conceptual classes. Construct a class diagram for the following scenario. [4+6] (2081 Chaitra)

Scenario: For a construction company, a software is to be developed. Company takes many projects at particular location. Each project is supervised by project manager assigned by CEO. Records related to start and completion of project are maintained. Under each project manager there is a team of people of different category like designer, plumber, electrician, architect, labor, etc. Each project is marketed by team of Marketing Executives.

Answer:

Category List for Airline Reservation Application:

Category	Conceptual Classes
Physical objects	Aircraft, Airport, Gate, Seat, Terminal
Transactions	Reservation, Booking, Cancellation, Payment, Refund
Roles of people	Passenger, Pilot, FlightAttendant, GroundStaff, TravelAgent
Places	Airport, City, Country, Gate
Events	Flight, Departure, Arrival, Delay
Specifications	FlightSchedule, FareClass (Economy/Business), SeatType
Organizations	Airline, AirportAuthority
Records of finance	Invoice, Ticket, BoardingPass, PaymentReceipt
Other systems	PaymentGateway, CheckInKiosk, BaggageSystem

Class Diagram for Construction Company — Classes and Relationships:

Class	Attributes	Key Relationships
Company	companyID, name, address, phone	Employs CEO, undertakes Projects
CEO	empID, name, designation	Assigns ProjectManagers to Projects
Project	projectID, name, location, startDate, endDate, status	Has 1 ProjectManager, located at 1 Location, has many Workers
ProjectManager	empID, name, experience, assignedDate	Manages 1..* Projects, leads 1..* Teams
Location	locationID, address, city, country	Site of 0..* Projects
TeamMember	empID, name, category, wage	Works on 1..* Projects (superclass for all worker types)
Designer	specialization, portfolioURL	Subclass of TeamMember
Electrician	licenseNo, voltage_rating	Subclass of TeamMember
Architect	licenseNo, certificationLevel	Subclass of TeamMember
MarketingExecutive	empID, territory, salesTarget	Markets 0..* Projects
MarketingTeam	teamID, teamName	Composed of 1..* MarketingExecutives, markets Projects

Key Associations in Class Diagram:

- Company 1 -- 1 CEO (Company employs one CEO)
- CEO 1 -- 0..* ProjectManager (CEO assigns managers)
- Project 1 -- 1 ProjectManager (Each project has one manager)
- ProjectManager 1 -- 1..* TeamMember (Manager leads team)
- TeamMember -- Designer, Electrician, Architect (Generalization/Inheritance)
- Project 0..* -- 1..* MarketingTeam (Project marketed by team)
- MarketingTeam -- 1..* MarketingExecutive (Composition)

SECTION 2: Adding Associations and Attributes to Domain Model

Exam Ref: 2080 Chaitra Q3, 2081 Chaitra Q4, 2079 Ashwin Q3, 2079 Jestha Q5, 2078 Baishakh Q3, 2077 Chaitra Q3 [2+2+6], 2074 Bhadra Q4

Q5. What do you mean by Domain Modeling? Present the guidelines to add attributes and associations in the domain model. [6] (2072 Ashwin)

Answer:

Guidelines for Adding Associations to Domain Model

An association is a meaningful relationship between two conceptual classes that needs to be remembered or recorded by the system. NOT every possible relationship needs to be shown — only those that the system must remember.

When to Add an Association:

- The association represents a relationship that must be preserved in the system (e.g., Sale records SaleLineItems).
- Derived from the phrase 'must-know' — object A must know about object B for the system to work.
- Corresponds to navigational need in the system operations.

Common Association Types (UML Notation):

Association Type	Symbol	Meaning	Example
Association	Solid line ——	Basic structural relationship — A and B are related.	Student -- Course (Student takes Course)
Aggregation	Hollow diamond <>——	Weak 'whole-part'. Part exists independently.	Library <>-- Book
Composition	Filled diamond *——	Strong 'whole-part'. Part cannot exist alone.	Order *-- OrderLine
Generalization	——> (hollow triangle)	Inheritance 'is-a' relationship.	SavingsAccount --> BankAccount
Dependency	- - - >	Temporary use/knowledge relationship.	OrderReport - - -> Order

Multiplicity Notation:

Notation	Meaning	Example
1	Exactly one	Order has exactly 1 Customer
0..1	Zero or one (optional)	Employee has 0..1 ParkingSpot
1..*	One or more (mandatory, many)	Order has 1..* OrderLines
0..* or *	Zero or more (optional, many)	Customer makes 0..* Orders
m..n	Between m and n	Team has 3..10 Members

Guidelines for Adding Attributes to Domain Model

An attribute is a logical data value of a conceptual class. It should be a simple value — not a complex object that should be a separate class.

Rules for Attributes:

- Use primitive/simple data types: String, Number, Date, Boolean, Text — NOT complex objects.
- If an attribute has its own attributes, or needs operations, make it a separate class.
- Do NOT use foreign keys as attributes (OO modeling — not relational modeling).
- Include only attributes relevant to the system's use cases — do not over-model.
- Avoid derived attributes (computed from other attributes) in domain model.

When NOT to use Attribute (use a class instead):

Wrong (Attribute)	Correct (Separate Class)	Reason
Sale.cashierName (String)	Sale --> Cashier (class)	Cashier has its own data: name, ID, shift, etc.
Flight.airportName (String)	Flight --> Airport (class)	Airport has its own: code, city, terminals.
Customer.addressString (String)	Customer --> Address (class)	Address has parts: street, city, postcode, country.
Product.supplierName (String)	Product --> Supplier (class)	Supplier has: name, contact, payment terms.

Q6. List out the possible ways to find concept name and relationships of domain model and construct domain model of Point of Sale Terminal of retail store that should include the concept of user management, inventory management, sale and payment. [2+2+6] (2079 Jestha)

Answer:

Ways to Find Concept Names (Conceptual Classes):

- Category List Method: Use predefined checklist of categories to identify classes.
- Noun Phrase Identification: Extract all nouns from problem description as candidate classes.
- Use Case Analysis: Examine each use case to find participating objects.
- CRC Cards: Class-Responsibility-Collaboration cards used in group brainstorming.

Ways to Find Relationships (Associations):

- 'Must-remember' criterion: If the system must remember the link between A and B, there is an association.
- Verb phrase analysis: Extract verb phrases from domain description (e.g., 'Customer makes Sale').
- Common association patterns: Knows/Has/Uses/Records/Manages/Is-a patterns.

Domain Model — POS Terminal (User Mgmt, Inventory, Sale, Payment):

Class	Key Attributes	Key Associations
Register	registerID, location	Records Sale, managed by Cashier
Sale	saleID, date, time, total, isComplete	Contains 1..* SaleLineItems, associated with Payment, recorded by Register
SaleLineItem	quantity, lineTotal	Described by ProductDescription
ProductDescription	productID, description, price, barcode	Describes SaleLineItems, part of Inventory
Inventory	inventoryID, location, lastUpdated	Holds 0..* ProductDescriptions
Payment	amount, paymentType, changeGiven, date	Associated with 1 Sale
Cash Payment	cashTendered	Subtype of Payment
Card Payment	cardNumber, authCode	Subtype of Payment
Cashier	cashierID, name, shift, loginTime	Operates Register, makes Sale
Manager	empID, name, access_level	Manages Store, oversees Inventory
Customer	customerID (optional), loyaltyPoints	Makes Sale
Store	storeID, name, address	Contains Register, has Inventory

SECTION 3: CRC Cards — Class-Responsibility-Collaboration

Exam Ref: 2082 Q10d, 2081 Q10b, 2080 Ashwin Q8c, 2079 Ashwin Q8d, 2078 Poush Q11d, 2077 Chaitra Q9a, 2074 Bhadra Q7, 2072 Ashwin Q8d

Q7. What is the purpose of CRC Cards as a tool for analysis? Justify the statement that 'Coupling and Cohesion are the two sides of a coin'. Explain their interconnection with the help of an example. [4+5] (2080 Chaitra)

Answer:

CRC Cards — Class-Responsibility-Collaboration

CRC Cards (Class-Responsibility-Collaboration Cards) are a brainstorming tool used in OOA to identify classes, their responsibilities, and their collaborations with other classes. Each card represents one class and is physically an index card.

Structure of a CRC Card:

Section	Content	Description
Class Name (Top)	Class name (e.g., Sale)	The name of the conceptual class.
Responsibilities (Left)	What the class KNOWS and DOES	Attributes (knows) and operations/behaviors (does) of the class.
Collaborators (Right)	Which classes it works with	Other classes this class sends messages to or receives from.

Example — CRC Card for 'Sale':

Class: Sale	
Responsibilities	Collaborators
Knows: date, time, total, status	SaleLineItem
Knows: list of line items	Payment
Does: calculate total	Cashier
Does: add line item	ProductDescription
Does: make payment	Register

Purpose of CRC Cards:

- Identify classes early in analysis phase through collaborative team sessions.
- Discover responsibilities: what each class must know and do.
- Find collaborations: which classes work together (reveals associations).
- Low-tech, easy to use — index cards allow physical rearrangement.
- Encourages thinking in objects rather than procedures.

Coupling and Cohesion — 'Two Sides of a Coin'

Coupling and Cohesion are the two most fundamental quality metrics in OO design. They are called 'two sides of a coin' because you must balance both — you cannot focus on one while ignoring the other. A good OO design always aims for LOW coupling AND HIGH cohesion simultaneously.

Aspect	Coupling	Cohesion
Definition	Degree of interdependency between classes/modules.	Degree to which elements within a single class belong together.
Goal	LOW coupling (minimize dependencies)	HIGH cohesion (keep related things together)

Bad case	HIGH coupling — changes in one class cascade to many others.	LOW cohesion — class does too many unrelated things.
Analogy	How many wires connect different appliances together.	How focused and purposeful one appliance is.

Example — Illustrating the Balance:

- BAD DESIGN: A single 'God Class' that handles user login, order processing, payment, AND inventory. This has LOW cohesion (does too much) AND HIGH coupling (everything depends on it).
- GOOD DESIGN: Separate classes — AuthService (handles login only), OrderService (handles orders only), PaymentService (handles payments only). Each has HIGH cohesion. Communication via well-defined interfaces gives LOW coupling.
- Connection: When you properly apply HIGH cohesion (single responsibility per class), you naturally achieve LOW coupling because classes only interact on focused, well-defined boundaries.

SECTION 4: Representation of System Behavior — System Sequence Diagrams (SSD)

Exam Ref: 2081 Chaitra Q5 [4+4], 2081 Ashwin Q4, 2080 Ashwin Q5 [6+6], 2079 Ashwin Q4, 2079 Chaitra Q4, 2078 Baishakh Q4 [8], 2077 Chaitra

Q8. Draw System Sequence Diagram (SSD) and Activity diagram from the given scenario. [4+4] (2081 Chaitra) Scenario: Customer arrives at a POS checkout with goods and/or services to purchase. Cashier starts a new sale. Cashier enters item identifier. System records sale line item and presents item description, price, and running total. Cashier repeats steps 3-4 until done. System presents total with taxes calculated. Cashier tells Customer the total and asks for payment. Customer pays and System handles payment.

Answer:

System Sequence Diagram (SSD) — POS Process Sale:

In an SSD, the Actor (Cashier) is on the left, the System (:POSSystem) is on the right. Solid arrows show system input events (actor → system). Dashed arrows show system responses (system → actor).

Step	Event (Cashier → System)	Response (System → Cashier)	Notes
1	makeNewSale()	OK — new sale created	Cashier starts the transaction
2 (loop)	enterItem(itemID, qty)	itemDescription, price, runningTotal	Repeated for each item scanned
3	endSale()	totalWithTaxes	Cashier signals no more items
4	makePayment(cashTendered)	changeDue, receiptPrinted	Customer pays, system calculates change

SSD Notation Elements:

- System Input Event: Named in the format operationName(parameter) — e.g., enterItem(itemID, qty).
- Loop Frame: [loop, for each item] encloses repeated events.
- Guard Condition: [cashTendered >= total] — condition for alternative flows.
- System Boundary: The System is treated as a black box — only external inputs/outputs are shown.

Q9. Draw Use case diagram, Activity diagram and Sequence diagram for the Reservation System scenario. [6+6+6] (2081 Ashwin Back) Scenario: The Reservation System is a vital component for businesses in the hospitality industry, facilitating the seamless booking of services or resources. Users interact with the system to make reservations for hotels, restaurants, or event venues. The system's architecture involves the Reservation Engine, Availability Checker, Customer Database, and Notification System. Upon a user's request for a reservation, the system orchestrates a series of events to check the availability, update the reservation status, and notify the user.

Answer:

Actors and Use Cases — Reservation System:

Actor	Use Cases
Customer/User	Make Reservation (<<include>> Check Availability), View Reservation, Cancel Reservation, Receive Notification
Admin/Staff	Manage Availability, View All Reservations, Generate Reports
Notification System (External)	Send Confirmation Email/SMS
Payment Gateway (External)	Process Payment

System Sequence Diagram — Make Reservation:

Step	Event/Response	Notes
1	Customer → System: searchAvailability(date, serviceType)	User specifies date and type
2	System → Customer: availableSlots[ ]	System returns available options
3	Customer → System: makeReservation(slotID, customerDetails)	Customer selects and confirms
4	System → ReservationEngine: createReservation(details)	Internal — creates record
5	System → AvailabilityChecker: updateAvailability(slotID)	Internal — marks slot as taken
6	System → CustomerDatabase: storeCustomerRecord(customerDetails)	Internal — saves customer
7	System → Customer: reservationConfirmation(reservationID)	Confirmation returned to user
8	System → NotificationSystem: sendNotification(customerEmail, details)	Email/SMS sent to customer

Q10. Prepare System Sequence Diagram (SSD) for Restaurant Order System with necessary assumptions. Explain the process of transformations from OOA to OOD. [6+4] (2079 Ashwin Back)

Answer:

SSD — Restaurant Order System:

Assumptions: Customer orders at table. Waiter uses tablet system. Kitchen prepares order.

Step	Event/Response	Actor/System
1	openTable(tableNo): tableOpened	Waiter → System
2 (loop)	addOrderItem(itemID, qty, notes): itemAdded, runningTotal	Waiter → System (for each item)
3	submitOrder(): orderConfirmed, orderID	Waiter → System
4	System → Kitchen: prepareOrder(orderID, items)	System → KitchenSystem
5	Kitchen → System: orderReady(orderID)	KitchenSystem → System
6	System → Waiter: notifyOrderReady(tableNo, orderID)	System → Waiter
7	requestBill(tableNo): billDetails, total	Waiter → System
8	makePayment(tableNo, amount, paymentType): receipt	Waiter → System
9	closeTable(tableNo): tableClosed	Waiter → System

OOA to OOD Transformation

The transformation from OOA (Object-Oriented Analysis) to OOD (Object-Oriented Design) is the process of converting the conceptual model of the problem domain into a design model that can be directly implemented. This is sometimes called 'from domain to design'.

Step	OOA Artifact	OOD Artifact	Description
1	Domain Model (Conceptual Classes)	Design Class Diagram	Conceptual classes become design classes with methods added.
2	Use Cases + SSD	Interaction Diagrams (Sequence/Collaboration)	SSDs evolve into detailed interaction diagrams showing object method calls.

3	Conceptual Class Responsibilities	Method Signatures	Responsibilities become specific methods with parameters and return types.
4	Associations in Domain Model	Navigability & Visibility	Associations get direction (navigability) and visibility specifications.
5	N/A	GRASP Patterns Applied	Controller, Creator, Information Expert patterns guide assignment of responsibilities.
6	N/A	Interface & Implementation	Abstract interfaces and concrete classes defined for extensibility.

SECTION 5: System Contracts & Operation Contracts

Exam Ref: 2077 Chaitra Q4 [2+2+4], 2078 Baishakh Q4, 2079 Jestha Q4, 2075 Baisakh Q2 [2+6]

Q11. What is operation contract? Construct a System Sequence Diagram (SSD) for online examination system with necessary assumptions. [2+6] (2075 Bhadra)

Answer:

Operation Contract — Definition

An Operation Contract is a formal document that describes the behavior of a system operation in terms of pre-conditions (what must be true before the operation) and post-conditions (what becomes true after the operation). It supplements the use case description and SSD by precisely specifying what each system operation does to the system state.

Structure of an Operation Contract:

Component	Description	Example
Operation Name	The system operation identified from SSD.	enterItem(itemID: ItemID, qty: Integer)
Cross References	The use case(s) this operation appears in.	Use Case: Process Sale
Pre-conditions	Assumptions about the state of the system/objects before the operation.	A Sale is underway (sale object exists and is active).
Post-conditions	Declarations of state changes after the operation — using past tense.	A SaleLineItem was created. SaleLineItem.quantity was set to qty. SaleLineItem was associated with the current Sale.

Example Operation Contract — enterItem():

Field	Value
Operation	enterItem(itemID: ItemID, qty: Integer)
Cross References	Use Case: Process Sale
Pre-conditions	A Sale is underway.
Post-conditions (past tense)	- A SaleLineItem sli was created (instance creation). - sli.quantity was set to qty (attribute modification). - sli was associated with the current Sale (association formed). - sli was associated with a ProductDescription, based on itemID match (association formed).

SSD for Online Examination System:

Assumptions: Student logs in, selects exam, answers questions, submits, views results.

Step	Event	Response
1	login(studentID, password)	loginSuccess, studentProfile
2	selectExam(examCode)	examDetails (title, duration, totalMarks)
3	startExam(examCode)	examStarted, questions[ ], timer started
4 (loop)	answerQuestion(questionID, selectedOption)	answerRecorded
5	submitExam(examCode)	submissionConfirmed, score, resultDetails
6	viewResult(examCode)	detailedResults, gradeCard

Q12. Describe the importance of system contract and describe pre-condition and post-condition of contract. Construct a system contract for any one system operation with respect to the Health Care case study. [2+2+4] (2077 Chaitra)

Answer:

Importance of System Contracts

- Precise Specification: Precisely defines what each system operation does without specifying HOW (implementation-independent).
- Design Guide: Post-conditions directly tell designers what objects to create, modify, and associate.
- Testing Basis: Post-conditions become test assertions for unit and integration testing.
- Communication: Provides unambiguous documentation for all team members.
- Reduces Ambiguity: Supplements use cases which describe WHAT happens, not the exact state changes.

Pre-conditions vs. Post-conditions:

Aspect	Pre-conditions	Post-conditions
Definition	Assumptions about the system state BEFORE the operation executes.	Declarations about the system state AFTER the operation completes.
Tense	Present tense (current state assumption)	PAST TENSE (what changed) — this is important!
Purpose	Constraints that must be met for operation to work.	Specify state changes: instance creation, attribute modification, association formed/broken.
Example	'A Sale is underway and items have been entered.'	'A Payment was created. Sale.isComplete was set to TRUE. Payment was associated with Sale.'

System Contract — Health Care: arrangeAppointment(patientID, doctorID, date, time)

Field	Value
Operation	arrangeAppointment(patientID, doctorID, date, time)
Cross References	Use Case: Arrange Appointment with Physician
Pre-conditions	Patient is registered in the system. Doctor is available on the given date and time. No conflicting appointment exists.
Post-conditions	- An Appointment object was created (instance creation). - Appointment.date was set to date (attribute modification). - Appointment.time was set to time (attribute modification). - Appointment.status was set to 'Scheduled'. - Appointment was associated with Patient patientID (association formed). - Appointment was associated with Doctor doctorID (association formed). - Doctor's availability for that slot was updated to 'Booked' (attribute modification).

SECTION 6: Noun Phrase Identification & Case Studies

Exam Ref: 2074 Bhadra Q4, 2073 Bhadra Q2b, 2072 Magh Q2b, 2079 Chaitra Q4, 2078 Chaitra Q4

Q13. Identify the different classes, relationships and draw a Class Diagram for 'Library Management System'. [4+6] (2081 Ashwin) Scenario: Library Management System Software is used to monitor and control transactions in a library. This case study on the Library Management System gives us complete information and daily transaction done in a library. We need to record and retrieve the details of books in the library which mainly focuses on basic operations like adding a new book, a new member, searching books, facility to borrow books and returns.

Answer:

Step 1 — Noun Phrase Identification from Scenario:

- Library, Book, Member, Librarian, Transaction, BorrowRecord, ReturnRecord, Catalog, Fine

Step 2 — Classes and Attributes:

Class	Attributes	Methods
Book	bookID, title, author, ISBN, publisher, year, category, totalCopies, availableCopies	searchBook(), updateStock()
Member	memberID, name, email, phone, address, membershipDate, membershipType	borrowBook(), returnBook(), viewHistory()
Librarian	librarianID, name, email, phone, shift	addBook(), removeBook(), issueLoan(), acceptReturn()
Loan	loanID, issueDate, dueDate, returnDate, status	calculateFine(), renewLoan()
Fine	fineID, amount, paidDate, paymentStatus	calculateAmount(), markPaid()
BookCopy	copyID, condition, location (shelf)	checkAvailability()
Catalog	catalogID, lastUpdated	searchByTitle(), searchByISBN(), searchByAuthor()
Reservation	reservationID, reservationDate, expiryDate, status	cancelReservation()

Step 3 — Key Relationships:

- Book 1 -- 1..* BookCopy (Composition — copies cannot exist without book)
- Member 0..* -- 0..* Book through Loan (Association)
- Loan 1 -- 1 BookCopy (Loan is for one specific copy)
- Loan 1 -- 1 Member (Loan belongs to one member)
- Fine 0..1 -- 1 Loan (Fine optionally generated for late loan)
- Librarian manages Library (Association)
- Member 0..* -- 0..* Book through Reservation (Association)
- Catalog 1 -- 0..* Book (Aggregation)

Q14. For the case study given below identify all the classes, their relationships, attributes and methods. Also draw class diagram using standard UML Notation. [10] (2079 Ashwin) Scenario: You are assigned to create Instagram clone project. Identify all classes, their relationships, attributes and methods for each class.

Answer:

Classes, Attributes, and Methods — Instagram Clone:

Class	Attributes	Methods
User	userID, username, email, password, bio, profilePic, followerCount, followingCount, isVerified	login(), logout(), editProfile(), deleteAccount()

Post	postID, mediaURL, caption, timestamp, likesCount, location, hashtags[ ]	createPost(), deletePost(), editCaption()
Story	storyID, mediaURL, caption, timestamp, viewCount, expiresAt	createStory(), deleteStory(), viewStory()
Comment	commentID, text, timestamp, likesCount	addComment(), deleteComment(), editComment()
Like	likeID, timestamp	addLike(), removeLike()
Follow	followID, followDate	followUser(), unfollowUser()
Message	messageID, content, timestamp, isRead, mediaURL	sendMessage(), deleteMessage(), markRead()
Notification	notifID, type, message, timestamp, isRead	sendNotification(), markAsRead()
Hashtag	tagID, tagName, postCount	searchByHashtag()

Key Relationships — Instagram Clone:

- User 1 -- 0..* Post (User creates Posts — Aggregation)
- User 1 -- 0..* Story (User creates Stories — Aggregation)
- Post 1 -- 0..* Comment (Post has Comments — Composition)
- Post 1 -- 0..* Like (Post has Likes — Aggregation)
- Comment 1 -- 0..* Like (Comment has Likes — Aggregation)
- User 0..* -- 0..* User through Follow (Self-referential association — Follow)
- User 0..* -- 0..* User through Message (Self-referential — Direct Message)
- Post 0..* -- 0..* Hashtag (Many-to-many — Post tagged with Hashtags)

QUICK REVISION: Chapter 2 — Key Points Summary

Topic	Key Points	Expected Marks
Domain Model / Conceptual Model	Visual map of real-world concepts. Classes, associations, attributes. NO methods. Reduces representational gap.	4–6
Strategies to Find Conceptual Classes	(1) Category List Method — use predefined categories checklist. (2) Noun Phrase Identification — extract nouns from description. (3) Use Case Analysis.	4–6
Category List Method	15+ categories: physical objects, transactions, roles, places, containers, events, organizations, records, etc.	4–6
Adding Attributes	Use simple/primitive types only. If attribute has own attributes → make it a class. No foreign keys.	3–4
Adding Associations	'Must-remember' criterion. Include name and multiplicity (1, 0..1, 1..*, 0..*). Show aggregation/composition where appropriate.	3–5
CRC Cards	Class-Responsibility-Collaboration. Index cards. Left: responsibilities (know/do). Right: collaborators. Used in OOA brainstorming.	3–4
Coupling & Cohesion	LOW coupling + HIGH cohesion = good design. Coupling = inter-class dependency. Cohesion = intra-class focus.	3–5
System Sequence Diagram (SSD)	Shows actor-system interactions for one use case. System events (actor→system, solid arrow) + responses (system→actor, dashed).	6–8
Operation Contracts	Pre-conditions (what must be true BEFORE). Post-conditions in PAST TENSE (what changed AFTER): instance creation, attribute modification, association formed/broken.	4–6
OOA to OOD Transformation	Domain Model → Design Class Diagram. SSD → Interaction Diagrams. Responsibilities → Method signatures. GRASP patterns applied.	4–6
POS Case Study	Key classes: Sale, SaleLineItem, ProductDescription, Payment, Register, Cashier. SSD: makeNewSale → enterItem (loop) → endSale → makePayment.	6–8
Multiplicity	1=exactly one, 0..1=optional, 1..*=one or more, *=zero or more, m..n=range.	2–3